

```
This is my own string!  
This is my own string!
```

Kapitel 11

Objektorientering 3: mer om arv

11.1 Olika typer av arv

I förra kapitlet såg vi olika exempel på arv i Python. Olika former av arv har olika namn inom objektorientering:

1. Enkelt arv (eng. *single inheritance*): klass B ärver endast från klass A.
2. Mångnivåarv (eng. *multilevel inheritance*): klass B ärver från A och klass C ärver sen från klass B.
3. Hierarkiskt arv (eng. *hierarchical inheritance*): klass A är superklass till B, C, D...
4. Multipelt arv (eng. *multiple inheritance*): klass C ärver från *både* A och B (men de ärver inte från varandra).
5. Hybridarv (eng. *hybrid inheritance*): en mix av två eller fler typer av arv.

Vi såg även att man kan illustrera olika typer av arv med hjälp av klassdiagram. Förra kapitlet innehöll exempel på enkelt, mångnivå-, hierarkiskt och hybrid-arv, så vi ska nu titta på multipelt arv.

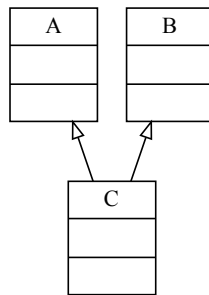
Det är inte alla programmeringsspråk som tillåter multipelt arv, men det gör Python. Betrakta följande exempel:

```
class A:
    pass    # some code

class B:
    pass    # some more code

class C(A, B):
    pass    # even more code
```

Vi kan illustrera detta med hjälp av ett klassdiagram:



Figur 11.1: Klassdiagram för A, B och C

Här har C alltså två superklasser, A och B, vilka inte ärver från varandra. Detta kan verka oskyldigt och man kan undra varför inte alla språk stödjer det. Men rent generellt är det lätt hänt att multipelt arv ger kaos på grund av så kallade *diamantproblem*.

11.1.1 Diamantproblem

Betrakta följande kod:

```
class A:
    def f(self):
        print("f of A called")

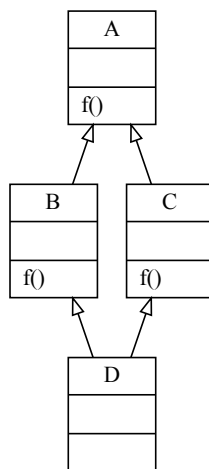
class B(A):
    def f(self):
        print("f of B called")

class C(A):
    def f(self):
        print("f of C called")

class D(B,C):
    pass

d = D()
d.f()
```

Detta kallas för ett diamantproblem då får man en romb om man ritat upp klassdiagrammet:



Figur 11.2: Klassdiagram för A, B, C och D

Kör vi koden ovan får vi

f of B called

Men om vi istället kör följande där arvet är D(C,B) får vi:

```

class A:
    def f(self):
        print("f of A called")

class B(A):
    def f(self):
        print("f of B called")

class C(A):
    def f(self):
        print("f of C called")

class D(C,B):
    pass

d = D()
d.f()
  
```

f of C called

Så utdata har nu ändrats då man ärver i en annan ordning! Denna typ av beroenden vill man helst inte behöva tänka på när man programmerar och därför tycker man att det är ett problem. Diamantproblem kan uppstå med multipelt arv. Man kan lätt föreställa sig hur kod snabbt blir oöverskådlig om man har

många klasser med en massa metoder och attribut vilka ärver från varandra på detta sätt. Denna typ av problem är vad som lätt orsakar kaos i kod med multipelt arv och varför många språk inte tillåter det.

11.2 Exempel: Algebraiska uttryck

Vi ska nu titta på ett till exempel på arv. Poängen med detta exempel är att illustrera två saker:

1. Hur man kan använda klasser för att *strukturera* data.
2. Hur man kan använda rekursion i kombination med klasser.

Denna kombination är väldigt kraftfull och låter oss lösa många olika typer av programmeringsproblem på ett enkelt sätt. Vi börjar med att skriva ett program för att representera och modifiera algebraiska uttryck. Dessa uttryck kan innehålla tal, variabler, +, * och *matchade* paranteser. Ett exempel på ett algebraiskt uttryck är $x * y + 7$ och ett annat är $x * (y + 7)$.

Hur ska vi tänka för att representera dessa i Python? En naiv lösning skulle kunna vara att helt enkelt använda strängar, så $x * y + 7$ skulle representeras som strängen " $x * y + 7$ ". Men låt oss säga att vi vill byta ut x till 3 och y till 2 och sedan beräkna värdet av uttrycket... Detta blir *väldigt* komplicerat med strängar!

Ett annat problem med strängar är hur man ska hantera paranteser. Enligt konventionen att * binder starkare än + så ska ju $x * y + 7$ tolkas som $(x * y) + 7$ och inte $x * (y + 7)$. Har man inte koll på detta kommer man ju få fel svar när man utvärderar uttryck. Ett mycket smidigare sätt är att använda *träd*. På så sätt kan vi representera algebraiska uttryck med två dimensioner istället för bara en!

Ett träd är en datastruktur där ett element antingen är ett löv eller en nod med ett antal grenar som leder till "underträd" (eng. *subtree*). Så $(x * y) + 7$ kan representeras entydigt som ett träd med översta noden + och två underträd som innehåller $x * y$ och 7. Värdet 7 är ett löv i trädet då det inte har några underträd. Uttrycket $x * y$ är även det ett träd, denna gång med noden * längst upp och x och y som löv. Vi kan rita detta som:

```
      +
     / \
    *   7
   / \
  x   y
```

Obs: dataloger ritar träd upp-och-ned jämfört med riktiga träd!

Uttrycket $x * (y + 7)$ är istället följande träd:

```
      *
     / \
    x   +
       / \
      y   7
```

Denna typ av träd kallas *binära* träd då alla noder har två eller noll underträd. Låt oss nu representera denna typ av träd med algebraiska uttryck i Python! Vi börjar med en tom superklass för uttryck som i sin tur har 3 subclasser (en för siffror, en för variabler och en för +):

```
class Expr:
    pass
```

```

class Constant(Expr):

    def __init__(self, value):
        self._value = value

    def __str__(self):
        return str(self._value)

class Var(Expr):

    def __init__(self, name):
        self._name = name

    def __str__(self):
        return self._name

class Plus(Expr):

    def __init__(self, left, right):
        self._left = left
        self._right = right

    def __str__(self):
        return str(self._left) + " + " + str(self._right)

print(Plus(Constant(3), Var("x")))

```

3 + x

Notera att anropet till str är *rekursivt* i __str__ för Plus. Här anropas str metoden för vänstra och högra underträdet. Vilken funktion som sen kommer köras beror på vad dessa är.

Låt oss nu lägga till *, detta kan göras genom:

```

class Times(Expr):

    def __init__(self, left, right):
        self._left = left
        self._right = right

    def __str__(self):
        return str(self._left) + " * " + str(self._right)

print(Plus(Times(Var("x"), Var("y")), Constant(7)))

```

x * y + 7

Men denna definition är ju exakt samma som Plus, bortsett från att vi skriver ut * istället för +. Kopiera och klistra in kod är farligt då man lätt introducerar buggar! En bättre lösning är att istället introducera en ny klass för binära operatorer vilken Plus och Times sen ärver ifrån:

```

class BinOp(Expr):
    def __init__(self, left, right):
        self._left = left
        self._right = right

class Plus(BinOp):
    def __str__(self):
        return str(self._left) + " + " + str(self._right)

class Times(BinOp):
    def __str__(self):
        return str(self._left) + " * " + str(self._right)

print(Plus(Times(Var("x"), Var("y")), Constant(7)))

```

```
x * y + 7
```

Detta verkar ju fungera bra, men det blir fel om vi skriver:

```
print(Times(Var("x"), Plus(Var("y"), Constant(7))))
```

```
x * y + 7
```

Vi får exakt samma som ovan! Detta är knas då vi glömt bort att sätta in paranteser. En naiv lösning på detta problem är att alltid sätta in parenteser runt underuttryck i Plus och Times:

```

class BinOp(Expr):
    def __init__(self, left, right):
        self._left = left
        self._right = right

class Plus(BinOp):
    def __str__(self):
        return "(" + str(self._left) + ") + (" + str(self._right) + ")"

class Times(BinOp):
    def __str__(self):
        return "(" + str(self._left) + ") * (" + str(self._right) + ")"

print(Plus(Times(Var("x"), Var("y")), Constant(7)))

print(Times(Var("x"), Plus(Var("y"), Constant(7))))

```

```

((x) * (y)) + (7)
(x) * ((y) + (7))

```

Detta är ju rätt, men vi har satt in onödigt många paranteser! Vad vi egentligen vill få ut är:

```
print(Plus(Times(Var("x"),Var("y")),Constant(7)))
```

```
print(Times(Var("x"),Plus(Var("y"),Constant(7))))
```

```
x * y + 7
x * (y + 7)
```

Hur kan vi lösa detta?

Lösning: dekorera alla uttryck med hur hårt de binder. Med denna extra information kan vi avgöra om vi ska skriva ut paranteser runt underuttryck eller inte. Vi gör detta genom att lägga till en klassvariabel *prec* (för *precedence*) till Expr klassen och sen skriva över den i Plus och Times. Vi har sedan en hjälpfunktion *parens* som avgör om paranteser ska skrivas ut runt ett uttryck eller inte.

```
class Expr:

    # default precedence very high so that we don't put parantheses
    prec = 1000

class Constant(Expr):

    def __init__(self, value):
        self._value = value

    def __str__(self):
        return str(self._value)

class Var(Expr):

    def __init__(self, name):
        self._name = name

    def __str__(self):
        return self._name

class BinOp(Expr):

    def __init__(self, left, right):
        self._left = left
        self._right = right

# Function for inserting parentheses around a string if precedence p1
# is smaller than precedence p2
def parens(p1,p2,s):
    if p1 < p2:
        return "(" + s + ")"
    else:
        return s
```



```

class Plus(BinOp):

    prec = 1

    def __str__(self):
        s1 = parens(self._left.prec, self.prec, str(self._left))
        s2 = parens(self._right.prec, self.prec, str(self._right))
        return s1 + " + " + s2

class Times(BinOp):

    prec = 2

    def __str__(self):
        s1 = parens(self._left.prec, self.prec, str(self._left))
        s2 = parens(self._right.prec, self.prec, str(self._right))
        return s1 + " * " + s2

print(Plus(Times(Var("x"), Var("y")), Constant(7)))

print(Times(Var("x"), Plus(Var("y"), Constant(7))))

print(Times(Var("x"), Times(Var("y"), Plus(Constant(3), Var("z")))))

x * y + 7
x * (y + 7)
x * y * (3 + z)

```

11.3 Exempel: Binära träd

Vi kan generalisera datatypen för algebraiska uttryck till godtyckliga binära träd. På detta sätt kan vi skriva våra funktioner en gång för alla och kommer inte behöva skriva om dem för varje typ av binära träd vi kan tänkas behöva.

Ett binärt träd är antingen en förgrening (med nodvärde och vänster/höger underträd) eller ett löv (med värde). Vi lägger in följande funktionalitet i dessa:

- `__str__`: konvertera till en sträng.
- `member(x)`: testa om `x` finns i trädet.
- `map_tree(f)`: applicera funktionen `f` på alla värden i trädet (notera att detta är en högre ordningens funktion då den tar en funktion som argument).
- `linearize()`: konvertera trädet till en lista.

I detta exempel är huvudklassen `BinTree` tomt, men det är ändå bra att ha med om man en dag vill lägga in något i den som vi gjorde med `prec` värdet för algebraiska uttryck.

```

class BinTree:
    pass

class Branch(BinTree):

    def __init__(self, node, left, right):
        self._node = node
        self._left = left
        self._right = right

    def __str__(self):
        n , l , r = str(self._node) , str(self._left) , str(self._right)
        return "(" + n + "," + l + "," + r + ")"

    def member(self, x):
        return self._node == x or self._left.member(x) or self._right.member(x)

    def map_tree(self, f):
        return Branch(f(self._node), self._left.map_tree(f), self._right.map_tree(f))

    def linearize(self):
        return self._left.linearize() + [self._node] + self._right.linearize()

class Leaf(BinTree):

    def __init__(self, val):
        self._val = val

    def __str__(self):
        return "(" + str(self._val) + ")"

    def member(self, x):
        return self._val == x

    def map_tree(self, f):
        return Leaf(f(self._val))

    def linearize(self):
        return [self._val]

t = Branch(-32, Leaf(2), Branch(1, Branch(23, Leaf(4), Leaf(-2)), Leaf(12)))

print(t)
print(t.member(12))
print(t.member(-123))

print(t.map_tree(lambda x: 0))
print(t.map_tree(lambda x: x ** 2))

```

```
print(t.linearize())
(-32,(2),(1,(23,(4),(-2)),(12)))
True
False
(0,(0),(0,(0,(0),(0)),(0)))
(1024,(4),(1,(529,(16),(4)),(144)))
[2, -32, 4, 23, -2, 1, 12]
```

Notera att alla funktioner som vi definierat är rekursiva. Detta exempel visar hur vi på ett enkelt sätt kan kombinera klasser och rekursion för att representera och modifiera strukturerad data.

11.4 Uppgifter

1. Vilka av exemplen i kapitel 10 är exempel på enkelt, mångnivå, hierarkiskt och hybrid arv?
2. Flytta `__str__` metoden för `Plus` och `Times` till `BinOp` på lämpligt sätt så att vi inte har någon kodduplikation. Exempen ska fortfarande skrivas ut på rätt sätt.
3. Lägg till en klass `Exp` för exponentiering av algebraiska uttryck. Den ska binda hårdare än både `+` och `*`, samt skrivas ut som operatoren `**` (dvs på samma sätt som i Python).
4. Lägg till en funktion `big_constant_sum(n,e)` där `n` är ett positivt heltal och `e` ett algebraiskt uttryck. Funktionen ska skapa ett uttryck där `e` adderas med sig själv `n` gånger. Om `n` är `0` så ska konstanten `0` returneras. Så funktionen ska alltså returnera summan

$$\sum_{i=0}^{n-1} e$$

5. Lägg till en funktion `big_sum(n,e)` där `n` är ett positivt heltal och `e` en funktion från heltal till algebraiska uttryck. Funktionen ska skapa ett uttryck som representerar

$$\sum_{i=0}^n e_i$$

6. Definera evaluering av algebraiska uttryck givet en uppslagstabell `d` från variabler till siffror. För att göra detta ska ni lägga till en metod `evaluate(d)` till `Constant`, `Var`, `Plus`, `Times` och `Exp`. Ni kan anta att alla variabler i uttrycket finns definierade i `d`.
7. † Implementera rosträd. Dessa är en typ av träd där varje nod innehåller ett värde och en lista med fler rosträd. Ett löv med värdet `5` representeras genom `RoseTree(5, [])`. Implementera en klass för rosträd som har samma metoder som `BinTree`.